

The Full Circle on Developer Productivity with Steve Yegge

Where does Steve's passion for developer tools come from? He talked through his career and learnings, and threw in a few stories that hadn't been previously shared anywhere.



GERGELY OROSZ

APR 11, 2023



191



16

Share



👋 Hi, this is [Gergely](#) with the monthly free issue of the Pragmatic Engineer Newsletter. In every issue, I cover challenges at Big Tech and high-growth startups through the lens of engineering managers and senior engineers.

If you're not a subscriber, you missed the issue on [Inside Figma's engineering culture](#) and [a few others](#). Subscribe to get two full issues every week. Many subscribers expense this newsletter to their learning and development budget. If you have such a budget, here's [an email you could send to your manager](#). 📌

[Steve Yegge](#) has been in the industry for more than 30 years, alternating between being a software engineer and an engineering manager. He worked at GeoWorks, Amazon, Google and Grab, and joined Sourcegraph as their Head of Engineering in October 2022. We recently did a deep-dive on how Sourcegraph operates in the issue [Inside Sourcegraph's engineering culture](#).

Steve became prolific within several tech communities thanks to his writing, which is casual, funny and covers topics that few have written about candidly. In 2008, he wrote the blog post [Get that job at Google](#). The blog post was one of the best preparation materials, at the time, for the Google software engineering interview. Coming from someone working at Google at the time, the advice was surprisingly candid.

Steve kept writing, although most of his writing stayed within the companies he worked at. Also in 2008 he wrote an internal rant at Google, explaining why, despite doing

almost everything better than Amazon, Amazon still does something much better than Google: building platforms. The “rant” gave a history lesson of the platform evolution at Amazon, cracked several jokes, and turned into a heavy criticism of Google. To give a taste of [the style of this rant](#):

“So one day Jeff Bezos issued a mandate. (...) His Big Mandate went something along these lines:

1. All teams will henceforth expose their data and functionality through service interfaces.

2. Teams must communicate with each other through these interfaces.

(...)

6. Anyone who doesn't do this will be fired.

7. Thank you; have a nice day!

Ha, ha! You 150-odd ex-Amazon folks here will of course realize immediately that #7 was a little joke I threw in, because Bezos most definitely does not give a shit about your day.”

I’ve been meaning to interview Steve right as I started my newsletter. However, back then, he was still retired. Since then, he came out of retirement, and I jumped at the opportunity to get to know more about Steve’s career and stories he has to share, and attempt to collect learnings that are both useful and inspiring for software engineers and engineering managers.

1. **Learning to code and GeoWorks.** Steve learned to code on a programmable HP scientific calculator. At his first job, at GeoWorks, he used the best debugger, to date—and that was 30 years ago! The experience left an impression on him, and an appreciation for developer tools.
2. **Amazon and the customer obsession mindset.** He joined Amazon a year after their IPO, became a technical program manager (TPM), and returned to a manager

role when he tried to go back to being an engineer.

3. **Google: 13 years at the company.** Steve started strong: shutting down Print Ads (the first project he joined) and upsetting people on the way. Leaving the company, he had some unlearning to do.
4. **Grab: accomplishing a lot, only with passion.** Steve's first trip at Grab led to Southeast Asia, to experience the service, on the ground. It was here that he considered that perhaps Google over-indexes on processes, which Grab had very little of, compared to the search giant.
5. **Retiring from corporate: the good and the bad.** Taking two years off from being an employee, Steve could get to a lot of the non-tech stuff he always wanted to do.
6. **Sourcegraph: bringing the customer-first mindset.** Observations about the company where Steve joined as Head of Engineering.
7. **The full circle on developer tools.** What started at GeoWorks, in 1992 - working with amazing developer tools - is coming full circle, with Steve.

With this, over to Steve:

1. Learning to code and GeoWorks

When I was 17, I decided I wanted to figure this programming thing out. I had a programmable HP scientific calculator, and this was the only computer in my household. So I used this device to teach myself 3D programming. I ended up rendering the [Utah teapot](#), which is a famous 30,000 node image from the '70s. It took half an hour to render on that HP calculator!

I was then at the Navy for a year, where I was a nuclear reactor operator. My dad kicked me out when I turned 17: he said that on my 18th birthday, I had to go and make it on my own. The Navy rolled out the red carpet for me, so that's the way I went.

Following my years in the Navy, I enrolled to study computer science at the University of Washington.

My first full-time job was at GeoWorks, building developer tools. GeoWorks was an Alameda-based company building an operating system. They came to campus in Seattle, Washington, in 1992, and made higher offers than any of the other companies recruiting from my university.

What really got me interested wasn't just the money, but how different they were to the other companies. When IBM recruiters showed up on campus, they told students to show up in suits for their interviews. At GeoWorks, there was none of this. They were hip, they were young, and they were one of the first truly "tech companies" - and I wanted to work for them.

GeoWorks built the GEOS 16-bit operating system in the '90s. This system was built in 8086 Assembly language, and the company was writing and maintaining millions of lines of Assembly code. Everything was Assembly: the operating system, the apps - you name it.

Tooling and developer productivity was incredible. I saw the best debugger I've ever used at GeoWorks. To this day, I've yet to see a debugger do what theirs did back then: path choice on the fly, undo on the spot, or step an instruction backwards. Even now, none of these fancy debuggers are as powerful as what we had there.

GeoWorks had a "Core Tools" team that was building world-class tools using extension languages like [Tcl](#). We also had [Emacs](#) gurus working there and I found it astonishing to watch how effective they are. That's where I started to learn a new Emacs thing every day, and I've been doing this for years.

These highly effective engineers at GeoWorks introduced me to the notion that developers can be super-productive.

We worked at a low level at GeoWorks. We didn't have "if" statements and we didn't have variables: we had registers. We used Assembly, and I also learned Tcl, Perl, shell scripting and Awk. That was pretty much everything we used there.

Even though we used Assembly, people were both super-happy and super-productive. It was all because of our tooling and our very basic CI/CD system. Developer tools made all the difference in feeling like GeoWorks is nirvana for software engineers.

You don't appreciate developer tools until you don't have them. I recently heard someone who left Google say:

"I didn't understand why Stevey was ranting all the time about developer tools, when he was at Google. Now I get it, after I left."

When you have great developer tools, it's nirvana. When you don't have them, either you don't know what you're missing, or you're lucky enough to work with some people who know what's missing, and so they evangelize getting those developer tools in place.

2. Amazon and the customer obsession mindset

I joined Amazon in 1998, a year after their IPO. Amazon was a very hot company back then to work for. I was going to join as an engineering manager, but they talked me into being a [technical program manager \(TPM\)](#). Even then, Amazon had this role.

So I joined as a TPM. I worked for [Kim Rachmeler](#), who was the group program manager for TPM. She was a master at cross-functional execution. Amazon's execution was world-class and TPMs were in the middle of it. This was when I learned the art of delivery.

These days, I recommend software engineers become a TPM at one point in their career. The TPM role exercises a muscle that we're all using, but not very well: the management of cross-functional projects. As TPMs, we worked not just with engineering but also with customer service, distribution center folks, and a lot of other areas.

Being a TPM gave me a tremendous amount of insight into the whole organization. And when, a year later, I decided it was time for me to leave the TPM role behind, it was easy for me to find my next team - because I'd already been working with so many other teams already! It was pretty cool how easy it was to move back to being an engineering manager, after I was a TPM. And so I chose to join the customer service team.

While I was a TPM, I fell in love with the customer service team and their problem space. Amazon had a lot of cool things going on, but the customer service challenge was unique in a way that I'd never seen before. Jeff Bezos said, right from the start: "Customer service is going to be [a profit center](#). We're not going to do QA (quality assurance): we'll just mix things up if it happens, and then we'll shower customers who had an issue with gift certificates and praise."

But how would the customer service tooling be able to answer any question that customers had, and fix any of their issues? It had to become this horizontal system that wraps around all other systems.

In retrospect, the need for world-class customer service tooling was the beginning for service discovery, because on the customer service tooling team we needed to be able to find and talk with all the systems that other teams built.

I tried to go back to being an individual contributor at Amazon, but got pulled back into engineering management shortly after. I've noticed that this keeps happening throughout my career: it happened at Amazon, and it happened later at Google as well. I would be working as an IC for a little while, and then someone comes up and asks, "Hey, could you take on just this little team?" More teams follow after the first one, and after a while I have 70-100 people in my organization. It's what happened at Amazon as well.

Why did I keep getting pulled back into engineering management? Well, first, you have to decide whether you even want to be a leader. Most people don't want to take on this role. Once you decide to give it a go, you've got to try it. There are a lot of people who thought they wanted to be a leader but ended up hating it.

As for me, I don't mind leading or managing. My problem is that I don't truly love the leadership aspect the same way as some people do. For me, leadership is a means to an end. Over the past 30 years, I've gotten okay at it, but it's not what gets me up in the morning.

What I get most excited about is the vision. It's the building part. It's the "Look, I'm going to do this by myself. Do you want to help?" feeling.

I'm good at pulling people with me to go and build something exciting. However, when it comes to the organizational side of things, I see myself as pretty average, and have to lean on my world-class support team to make it work.

In the end, at Amazon, I spent about two years in customer service tools, then moved back into developer tools. By the time I left, I was running about half of the developer tools organization.

The one thing I learned at Amazon that really helped me later was the customer obsession mindset. At Amazon, we were told that the customer is everything. This approach worked well for the company. And things later came full circle in a surprising way, when I was on the other side of this, when I worked at Grab.

Grab was a customer of AWS, and being this customer was an unbelievable experience. The AWS team sat down with me and asked things like: "What do you need? What can we do for you? Would elastic indexes help your business case?" When I say they sat down with me, I don't mean about an account executive or a customer agent. It was with the PM for DynamoDB, the engineering manager and the general manager (GM) for DynamoDB.

As they're talking to me, I'm now looking over my shoulder, trying to second-guess where the important person is in the room. After all, as Grab, we were still a tiny customer, compared to, for example, Netflix - one of the large Amazon customers. But it didn't matter, as Amazon treated their enterprise customers like royalty.

This is an experience I am absolutely bringing to Sourcegraph as we work with our enterprise customers. Treating customers very well was a huge takeaway for me.

Read more insights about the TPM role in the issue [What TPMs do and what software engineers can learn from them](#), and about the culture at Amazon in [Inside Amazon's engineering culture](#).

3. Google: 13 years at the company

I was happy at Amazon and had a lot of friends there. Things were going well. However, this strange thing happened. I kept losing more and more interns to Google.

At Amazon, I was a "Closer." If anyone has an offer from another company, and they were leaning towards that other company, they'd bring me in and I'd close them on Amazon. I had a pretty good track record on closing.

However, we started losing interns, despite me doing the closing calls. To me, this felt like a canary in the coal mine. Why weren't interns choosing Amazon?

So I started to dig in. What was it about Google that these interns were more excited about than what we were doing? And the more I dug into this, the more I realized that I also wanted to work at Google.

When joining Google, I decided to start as an individual contributor this time. It would be one of those cases where I'd eventually be pulled back into management again, but I lasted longer than in the past: I was an IC for six years.

At Google, I first worked on an Ads project, then a music search project, and then I did [Grok](#) - a large-scale source code analysis project.

I was an L7 at Amazon, and got hired as an L5 at Google. L7 at Amazon maps roughly to Principal Engineer and L5 at Google Maps to Senior Engineer. And I can count myself lucky, because some of my peers came in at an even lower level. Google has always had a high hiring bar and was strict on levelling, and this was especially true back then.

During my time at Google, I ended up getting two promotions, so I was L7 by the time I would leave. I probably could have been promoted more, if I'd decided to play the game. The thing is, it's extremely difficult to break into the Director level (L8) at Google. To do this, it becomes a numbers game at one point. You have to have an organization of a certain size.

Because getting to Director levels and your organization size are often connected, this leads to a lot of infighting. However, I was always giving teams away because I didn't care about the promotion. All my peers were directors, and I had an organization that was roughly the size of what directors would run. I just didn't care about the title.

I killed my first project at Google: Print Ads. I joined the Print Ads team, and within six months made the proposal to kill the project. I remember how Susan Wojcicki - who would later be the CEO of YouTube, from 2014 to 2023 - was particularly mad at me for doing this, because she thought that there was still a billion dollars lurking in this market. But I didn't.

As I joined the Print Ads project, I had to become an ads demand expert, and so I did. For six months I worked hard on how to make the project succeed. Then, I wrote a detailed postmortem outlining the decision tree on why Print Ads couldn't possibly work.

The postmortem went along the lines of, "This is everything that we could think of trying, and here's why it doesn't come together." The gist of the problem was that the industry feared and disliked Google, and the Google brand was the reason why Print Ads wouldn't take off.

After I left the team, Google tried on two more occasions to get Print Ads working. They asked for my postmortem document both times, before giving up.

You might find it strange that a software engineer could have this much impact: stopping an entire, potentially large initiative at a company like Google. But don't forget that this was during what I like to think of as the golden age of Google, around 2003-2008, when bottom-up innovation was happening everywhere.

Just to illustrate how bottom-up we were, after I started on the Print Ads in the Seattle office, Larry and Sergey made a joint trip to this office. Back then, the Seattle office was tiny, and only one floor. Larry and Sergey walked around, and when they got to our desks, they were like: "Hey, you're the Publication Ads team? Are you getting the support you need out of the sales and product teams?"

We had the PM and the engineering manager around us as well, but they were looking at me, as I was the tech lead of the engineering team. And I told them that I didn't think we were getting the support because the sales teams wanted to go after the big customers, and we wanted to go after whoever could become long-term customers.

Larry then said: "Well, we want you to treat this like a startup. You're running this. The engineering team does what the engineering team should do."

They said all of this while also looking at the PM and the EM, who had previously been complaining that we - the engineering team - weren't working on what they wanted us to do. They sent a very clear signal that things were working bottom-up.

Leaving Google, one thing I had to unlearn was creating barriers that prevented engineers from talking to customers. If you are someone working at Google, this is something you can perhaps help fix. The really big issue that I now see is really hurting Google right now is that they've put so much process - and so many barriers and obstacles - in place that this gets in the way of engineers talking with customers. This is especially the case with Cloud. The engineers have no idea what the customers want.

Google's engineering operates in a different plane of existence. They tend to think, "How can we get this wonderful experience we have inside Google, to reach our customers?" However, this is a huge mismatch to what customers want. I don't think Google has solved this mismatch and needs to listen to their customers more.

4. Grab: accomplishing a lot, only with passion

When I joined Grab, I travelled to Southeast Asia to meet the team and experience their products first-hand. The experience on the ground was so wild that it's hard for me to characterize it. It's like when Anthony Bourdain said that Tokyo is the world's biggest party and you're not invited. It felt like that.

Southeast Asia is just exploding. Just being there, in the center of all of it, was amazing.

Then, I talked to actual customers - to some of the drivers and couriers. They would tell me how they were able to buy an Android phone for their kids, or a household utility, thanks to their earnings, and this was super-inspiring to see: how our product was making a difference.

Being out there, in Southeast Asia, changed the way me and my team thought about things: both from a design perspective and from a technical one. Customers and

drivers were legitimately confused about things that we thought should have been obvious.

I ended up partially working on developer tooling at Grab as well. A person on my team built out a bunch of machine learning infrastructure to support personalization functionality. As part of this, I worked closely with the dev tools team at Grab there - to the point that this team asked me to become an honorary member of their team!

What did my title of 'Head of Engineering' at Grab mean? It was akin to the Director of Engineering role.

I'm observing a move away from the Director title, across the industry, because this title tends to be more controversial. "Head of" better conveys the intent of the title: that this person is the leader of that space. Also, it makes it clear that you don't need to be at a particular level to have this title: which is not true for Director. At Grab, Head of Engineering spans all the way from levels 7 to 9.

One learning from Grab was how much you can accomplish with just passion. At Grab, the team was pretty junior. Of course, there were some very senior folks as mentors and guides, but the majority of the people were less experienced.

I think that Google over-indexes on things like code health, processes, and repeatability. Contrast this with Grab, and companies similar to Grab. Those scaleups get stuff done in a scrappy way, and I admire how they do this.

Ultimately, I realized that it was a mistake to sign up for a job that was 17 time zones away. I got to the point where I was travelling eight times a year to Southeast Asia, pulling 13 to 14-hour days for months and it simply wasn't sustainable. I learned that if you need to lead teams in Asia, go to Asia.

This was the main reason I ended up leaving Grab. I left in 2020 and knew that I needed a long-term recharge.

5. Retiring from corporate: the good and the bad

For the most part, I just laid and recovered during the pandemic. I worked on a side project of mine - a game called [Wyvern](#) - and made some nice progress. However, really, I just needed a break.

Once I finished recovering, I started getting antsy, thinking about all the kinds of things I could be doing. I did [my YouTube channel](#) for fun, as an experiment. My favorite episode of the ones I recorded is [the one on Emacs](#). I loved how many people it resonated with. I don't think people realized that I was showcasing the same experience I had back in 1992, when I observed software engineers at GeoWorks using Emacs. People were commenting, saying, "What, you can do that??" It was so cool to get this reaction.

Because we were in the middle of the pandemic, I didn't have much to do. Had it not been for this, I would have had a lot of fun going out, meeting people, and travelling around this time. As the pandemic has eased, I have been making up for lost time and grabbing coffee with lots of people, though.

Then, the founders of Sourcegraph, Quinn and Beyang, reached out to me through Maxim Fateev, who is the cofounder of Temporal. I knew Maxim because he was friends with Nikolay Sokratov, whom I worked with at Grab. As I talked to them, I realized I had unfinished business around developer tooling, and it was a no-brainer for me to join.

What were the good and bad things about my "retirement?" The good thing was that I needed the break. I also had enough non-tech stuff to do that it felt to me that it was time to start with all of that. I was pretty happy, overall.

What was not so good is how "retiring" made me a bit lazy. Since I "un-retired," I got back to the person that I was before. I'm passionate and excited, working hard. I get fired up just by coming back to work on something that's fun. I can now see this difference.

Honestly, I doubt any company other than Sourcegraph could have pulled me out of retirement.

6. Sourcegraph: bringing the customer-first mindset

Why did I join Sourcegraph? After all, I'd worked at places like Amazon and Google before. Could a company become as big as Google has?

Personally, I'm skeptical that any company will be able to grow to Google's size, once the FTC passes new anti-trust regulation. So we might have already lived through a unique period where we had these giant tech companies, right? However, there will be new tech giants. We always think that we're done, but there are new companies which are born, and which grow.

Developer tools is an area that's is not understood well enough, anywhere. This is the one theme that's persisted through my career: I kept getting back to working on dev tools, again and again.

Sourcegraph is surprisingly open. Their model isn't that different from that of DataStax or similar companies. It's open source with an enterprise SaaS model.

Where Sourcegraph takes the part of being open a lot further than any other company is how they're open about their corporate practices and their philosophies and values. A lot of these are spelled out in great detail [in their handbook](#) - which is also in the open. This handbook contains things that most other companies would consider proprietary and confidential.

Because of this open nature, we're able to talk in the open about our planning process, thought process, and even strategic processes. Other companies might go, "Oh gosh, what if somebody fast-follows my strategy first and we lose?" At Sourcegraph, we don't worry about this because we're in it for the long game. We know that by being open, we're going to win, and we'll win partially because of this openness.

How do we bring the customer-first mindset to Sourcegraph? The good news is that it's already happening. But we're also looking into programs where engineers get to work directly with a customer. I'd like to model how Amazon and Grab both handled their customer engagement, above a certain employee seniority level. For example, at

Grab, at the senior engineer and above levels, we of course took the Grab rides, ordered the Grab food, and used Grab pay. But we also had to sit down with the drivers and talk with them about their experience.

At Amazon, as engineers, Bezos would send us off to call centers and watch the customer support representatives answering customer emails. I'd sit there and panic, thinking we need to fix that bug that I saw our system do. This was because it was painstaking to see the representative have to work around a small bug: that small bug causing a lot of headache for them.

My old boss, Mark Porter, was the CTO at Grab, and he said something I still remember:

"You know those *rough* edges of our product we always talk about? When you're a customer, they become *sharp* edges."

So why do most companies not expose software engineers to customer support?

It's a proven recipe that exposing engineers to customers works, after all. It worked very well for Amazon, and it worked for Grab too.

But here's the thing. Companies generally have a budget in how much they invest into things that they really don't want to do. Take, for example, interviewing. Most companies conduct pretty standard interviews, and know that they don't get very good signals from here.

A company that interviewed differently, and had a very high hiring bar, was GeoWorks. GeoWorks required people to do a six-month callout before getting a full-time offer, meaning it could gather more signal than most companies could dream of gathering, before offering the candidate a full-time position.

Most companies, however, under-invest in things. They under-invest in hiring and they under-invest in things like developer tools.

You can get ahead of your competition by breaking out from what everyone else is doing. For example: invest in engineers meeting with customers. Let's say they do this two weeks every year. Sure, they'll take a productivity hit, but they'll build up empathy

for customers. The problem starts to become how we're not good at measuring intangible results such as these.

What things am I excited about, at Sourcegraph? For code search, I think that Sourcegraph is already a lot better than anything else out there. With code intelligence, we're connecting dots that previously weren't connected.

The intelligence platform is one that will power better code reviews and enable massive refactorings to be done quickly. It's one that will power lightweight IDEs (integrated development environments).

Take Google as an example of investing in code intelligence. The developer tooling ecosystem at Google is huge, and yet it's likely Google doesn't have more than a handful of people working on this problem space. Compared to how much effort we'll spend on this area at Sourcegraph, Google's investment looks minimal. And it makes sense for Google - because the decision maker deciding on the budget will be constrained.

Taking a step back: code has become "Big Data." This is a broader trend that I'm seeing. We have customers that have hundreds of thousands of git repositories. They have a data management nightmare. Code has become as bad as data lakes were 5-10 years ago, and this problem space is on fire.

Companies solving this problem will rise to the occasion. So many tech companies are struggling with problems thanks to the scale of their codebase. And IDEs are not helping solve this problem.

This is one reason why companies turn to us. And whatever we do, we'll prioritize based on the needs of our customers. We have this incredible code intelligence solution that's scalable. We'll figure out the right ordering of work we do, but we'll do it by working with customers.

Here's an example of something we'll eventually do. Wiring up the stack trace in the production log, so that it maps to the line of code that caused the outage, is something we did at Google. It was super-useful when it worked. This is something we have the capability to do, and something that would be really useful at most companies.

And then we're just scratching the surface. Think about the dynamic runtime data we could bring in, even to improve search quality. For example, could you add developers voting on search results as an additional user signal, or find other clever ways to turn all this data into something that's even more useful?

We're facing a vast and unexplored problem space with large codebases. As long as we think about this as a Big Data problem, I expect to see unbelievable innovation in this space.

So what comes after code intelligence? I'm gonna say the same thing I told people at Sourcegraph on my second day.

If you look at dev tools, they are tools. Let's call them hammers and saws. Well, Sourcegraph is a flashlight. You can turn this flashlight on and off, you can make it light a large area with weak light, and you can also make it tight and focused.

Right now, Sourcegraph is the unfocused and large version of this flashlight. Code intelligence is best of class with amazing coverage, but the experience is not nearly as good as if you're inside an IDE.

At Google, we turned this flashlight into a laser pointer and shined it around this dark cavern of unexplored space of code intelligence. We found interesting things that you'll also see coming out of Sourcegraph in the future.

While building new things, we had our share of misses on the way. For example, we'd build features like, "Maybe our users would want to query these cases" - and then nobody used that feature.

Still, I feel that most people think that we're done with IDEs, because IDEs have barely changed in 25 years. However, with the machine learning stuff that's coming out in code search, this is just the beginning. I'm not just talking about what we're seeing with GitHub Copilot, but all the things that will follow.

Code intelligence is going to transform the whole toolchain, and Sourcegraph will be at the center of this.

7. The full circle on developer tools

Nobody had done dev tools right, and only Google has gotten close to getting it right. Google's internal developer ecosystem is as close as it gets to getting developer tooling *almost* right. This is also why many former Google engineers go off to other companies to try and copy parts of it. However, most companies just can't get behind a Herculean effort like this, and they're asking: why? Why would we invest so much into dev tooling?

I realized that big companies are not capable of getting developer tooling right. Google is not capable, Amazon is not capable, and neither is Netflix. These companies can't disrupt themselves to build the type of world-class developer tooling the world would need.

Yes, these companies build really, really cool internal developer tools. But they appear incapable of budgeting to build anything for engineers *outside* of their company to be used. Nobody is doing this.

There's a working model for software like stream-processing platform Kafka, which was originally created at LinkedIn. Then LinkedIn open-sourced it, and other companies are working together in a way that benefits the whole industry.

Why has this model of collaborating and benefiting the whole industry not happened with dev tools? My take is that there's an integration component that makes this too challenging.

Integrations are really hard with dev tools. You need that code intelligence platform at its core, and then you need to do lots of integrations. For example, with source control you would need to integrate not just with Git but also with solutions like SVN, Mercurial, and others. You'd need to add support for a variety of programming languages - including ones your company might not use.

I think that most companies simply don't have an appetite for the long tail of integrations. However, a company like Sourcegraph has opted to go this way.

This is what made me bullish on Sourcegraph. Dev tools that benefit everyone are coming, and these can only be built by a company that doesn't shy away from the integration part. This is the type of solution that can win in this space.

One more thing that made me excited to join was how, at Google, as I worked on the Code Search ecosystem, I was waiting for someone to come and compete with us. Years later, I would have companies offer something comparable to what we had at Google. And yet, I haven't seen anyone, outside of Sourcegraph. So this made the decision to join easy.

I will keep coming back full circle on developer tools, until someone fixes this space. 31 years ago, at GeoWorks, I worked with some of the best developer tools, which were well ahead of the industry. This gave me a glimpse of what great tools would do.

And I've kept waiting for companies to step up and fix the developer tools experience - year after year, decade after decade.

I'm now done with waiting and I want to fix this space.

I'm now a decision maker at Sourcegraph and I decided that I'm going to go and build better tools for developers, and I have a much more generous budget to work with. As a code intelligence platform, Sourcegraph becomes an aggregator. I don't think we'll become the next Google, but we might become something akin to the next Atlassian or the next JetBrains when we're done with this work.

So if you're passionate about developer tools, you should stay tuned.

Takeaways

This is Gergely again.

Thanks for taking the time to talk, Steve. You can follow Steve [on LinkedIn](#), check out [his YouTube channel](#) with interesting talks, and read [his blog](#). Steve recently published his first blog post in a while about coding assistants, AI and LLMs with the title [Cheating is all you need](#) - and I recommend reading that one as well.

It's hard to convey through text just how full of energy Steve was, throughout our conversation. It was so clear that Steve not only loves the developer tools space and cares about it but also lives and breathes it.

Here are the key takeaways I picked up from talking with Steve:

Down-leveling was a risk worth taking midway into Steve's career. Steve was an L7 at Amazon, but got an L5 offer at Google. Back in the day, Google was the hottest startup amongst software engineers. Still, Steve *did* have 16 years of experience by this point. He sensed that Google had strong momentum - seeing interns consistently choose Google over Amazon - and put aside levels, joining at the level offered.

Of course, when it comes to accepting a down-level offer, there is no generic advice. If you can join the next Google, sure, this is likely a fair trade-off. The problem is it's hard to predict which company might be on such a growth trajectory! *We covered more on down-leveling in the issue [The Seniority Rollercoaster](#).*

Customer obsession, as a mindset, is incredibly powerful. Having worked at Amazon, in the early days of the company, Steve walked away with a strong appreciation of putting customers first. Over the decades, he continued to appreciate this approach even more, and it's one of the biggest changes he's brought to Sourcegraph, right as he joined.

Curiously enough, the thing that Steve said he had to unlearn was how Google created barriers for engineers to talk to customers. This observation was interesting to hear - and it's a reminder for every founder and manager that the closer engineers are to customers, the better the outcomes that could come from this.

Working too many time zones away can be too much. Steve is one of the most energetic people I've talked with. Still, he sounded weary when he was talking about the second part of his time at Grab - once the honeymoon period was over.

Turns out that there are some things that are very hard to work around. Being an engineering leader in a completely different time zone to where most of your team is can wear you out, and it wore Steve out.

Once you see the difference world-class developer tools make, you cannot unsee it. Steve got exposed to what was likely the bleeding edge of developer tools at GeoWorks: at his first job, out of college.

It felt to me that the reason Steve kept gravitating to the developer tools space throughout his career was because he knew that if he could recreate that GeoWorks' developer experience - in a modern setting - then that would make a huge difference to any and all organizations.

I hope you enjoyed the stories, and found learnings from Steve's experiences that you can reflect on. Good luck to Steve and the Sourcegraph team in moving developer tooling forward!

And if you'd like to learn more about how Sourcegraph works from an engineering point of view, and how their 'Transparent by default' principle shapes their culture: we went deep in the article [Inside Sourcegraph's engineering culture](#).

I have a favor to ask: I'm running the [Tech Leaders Compensation survey](#), together VC firm CREANDUM to answer the question "how are engineering leaders at tech companies compensated?" If you are in a lead or management position and have a few minutes to spare, [please share anonymous compensation details here](#). I'll share detailed results in the newsletter around June - or add your email to get the report even before.

This is a collaboration I suggested after reading an earlier report from CREANDUM on an [early-stage founders compensation report](#). I helped build the questions, and will also be analyzing results. I am not paid for this collaboration. For more details, see [my ethics policy](#).

Hire Faster With The Pragmatic Engineer Talent Collective

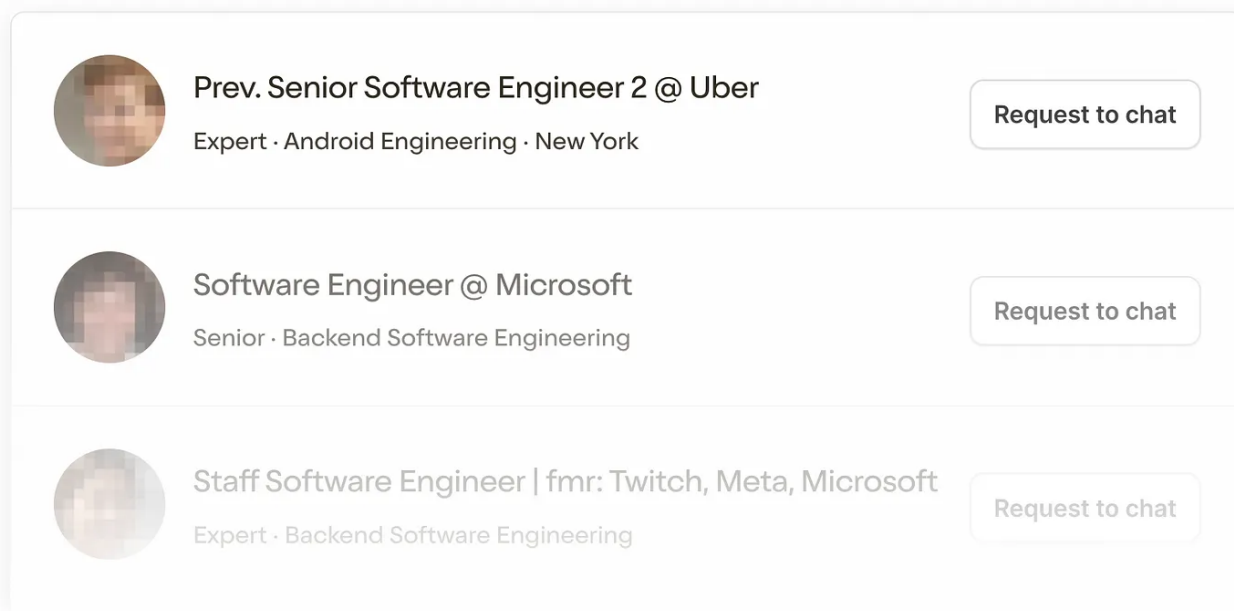
If you're hiring software engineers or engineering leaders, join [The Pragmatic Engineer Talent Collective](#). It's the #1 talent collective for software engineers and engineering

managers. Get weekly drops of outstanding software engineers and engineering leaders open to new opportunities. I vet every software engineer and manager - and add a note on why they are a standout profile.

Companies like Linear use this collective to hire better, and faster. [Read what companies hiring say](#). And if you're hiring, apply here:

1281 active candidates

Updated a day ago



HIRE CANDIDATES FROM TOP COMPANIES



If you're open for a new opportunity, join to get reachouts from vetted companies. You can join anonymously, and leave anytime:

Featured Pragmatic Engineer Jobs

1. [Engineering Leader - Card Platform](#) at X1 Card. **\$250K+**. Remote (US).
2. [Senior iOS Engineer](#) at Polarsteps. **£70-90K**. Amsterdam.
3. [Frontend Software Engineer](#) at Enveritas. **\$130-150K**. Remote (Global).
4. [Senior Android Engineer](#) at Polarsteps. Amsterdam.
5. [Senior Mobile Developer \(React Native\)](#) at Peppy. Remote (UK).
6. [Head of Data](#) at Peppy. Remote (UK).
7. [Senior Software Engineer, Distributed Systems](#) at Mixpanel. **\$200-270K** + equity. New York, San Francisco, Seattle or Remote (US).
8. [Senior Software Engineer, Fullstack](#) at Mixpanel. **\$200-270K** + equity. New York, San Francisco, Seattle or Remote (US).
9. [Senior Fullstack Engineer](#) at Synthesia. **£70-110K** . Remote (Europe or US Eastern time) or onsite (London, Amsterdam, New York).
10. [Senior Backend Engineer](#) at Synthesia. **£70-110K** . Remote (Europe or US Eastern time) or onsite (London, Amsterdam, New York).
11. [Senior Frontend Engineer](#) at (catch) Health. **\$90-120K** + equity. Remote (North America).
12. [Solutions Engineer](#) at Pigment. **\$70-120K**. New York or Toronto.
13. [Senior Backend Engineer](#) at Comun. New York or Remote (US).

See more senior engineer and leadership roles with great engineering cultures on [The Pragmatic Engineer Job board](#) - or post your own.



191 Likes · 16 Restacks

© 2023 Gergely Orosz • [Privacy](#) • [Terms](#) • [Collection notice](#)
~~Substack is the home for great writing~~